Do this next

Eleven things are waiting on a person rather than on code. They are in the order that gets the most settled for the least effort: the first three take about twenty minutes between them and answer questions that have been open since this project started.

Nothing here needs a developer. Where a screen's wording might have changed since this was written, the thing to look for is described rather than quoted.

New in October 2026: three more, ten minutes

These came with the October security update and the giving room. CHANGELOG.md (https://github.com/klydo131/open-sentry-beacon/blob/main/docs/../CHANGELOG.md) has the reasons.

1. **Shorten how long an email code lasts.** In Supabase, open **Authentication**, find the setting for how long an email sign-in code or link stays valid, and set it to **3600** seconds (one hour) or less. A code that lasts longer is longer to steal.
2. **Say yes to the two database updates** when you are asked: the giving room, and every table opened by name. Nothing is applied to the live database until you say so, and until then Home simply has no giving card. Giving is optional and starts off, so the update also asks whether to switch it on for your church. After that, a Director opens **Settings → Giving**, types the month's goal and the church's GCash or bank details, and names the treasurers. Members see nothing about giving until that is done.
3. **Delete the court room's e-mail function, if it was ever deployed.** The court room closed on 27 October 2026. In Supabase, **Edge Functions**, if a function called `door-code` is listed, delete it (or run `npx supabase functions delete door-code`). Keep the `BREVO_API_KEY` and `BREVO_SENDER` secrets: the invitations use them (EMAIL.md (https://github.com/klydo131/open-sentry-beacon/blob/main/docs/EMAIL.md)). Then, in **Storage**, delete the empty bucket called `court-exhibits`.

---

1. Find out what is actually deployed · 2 minutes

Every push for the last three weeks has ended with "pushed, build not observed", because the machine doing the work cannot reach the site. Everything else in this list is guesswork until this is done.

1. On your phone, open the app the way a member does.
2. Add `/version.json` to the end of the address and load it.
3. You will see a short line with a `built` date in it.

If the date is today or yesterday, the deploys are landing and you can tick this off for good.

If it is older than your last push, deployment is stuck. Go to the Vercel dashboard, open the project, look at the most recent deployment, and read its log. That log will say what failed.

---

2. Find the Vercel account, and what it costs · 5 minutes

This is the one number that could still put the church over budget, and it has been unknown for the whole project. The account this assistant can reach is

so the site is served from somewhere else.

1. Go to **vercel.com** and sign in with the account that deployed the app.
2. Top left, open the account switcher. Note the **name** of the team the project is under.
3. Open that team's **Settings** → **Billing**, and note the plan: **Hobby** or **Pro**.

Hobby is $0 and the costing stands. **Pro is $20 a month**, about ₱1,160, which puts the church over ₱2,000 again even after step 3 below.

> Tell me the team name and the plan and I will finish the costing. If it is on Pro, it is worth knowing that Vercel's Hobby plan forbids commercial use, so whether you can move back down is a licensing question rather than a technical one.

---

3. Stop paying for the project nobody uses · 3 minutes

Your Supabase organisation runs two projects. The second is the predecessor, with nine accounts on it, last signed into on 26 August. It costs about **$10 a month**, which is the entire amount by which the church is over budget.

1. Go to **supabase.com/dashboard** and sign in.
2. Open the project called **Local Church App Project**. Check you have the right one: it should have about nine accounts, not sixty-three.
3. **Project Settings** → **General**, scroll to the bottom, and choose **Pause project**.

Pausing keeps the data. You can restore it later from the same place.

Do not pause `Open-Sentry-Beacon`**.** That is the live church.

---

4. Switch on the daily keep-alive · 5 minutes

Two secrets, and the job starts working by itself.

1. Go to your GitHub repository → **Settings** → **Secrets and variables** → **Actions**.
2. Press **New repository secret**, twice:

Name	What to paste
`SUPABASE_URL`	Supabase → Project Settings → **Data API** → Project URL
`SUPABASE_ANON_KEY`	The same page, the **anon public** key

Copy them exactly, with no spaces at either end.

1. Go to the **Actions** tab, choose **keep-awake** on the left, and press **Run workflow** to test it now rather than waiting for tomorrow.

A green tick means it worked. It then runs itself every day at 9pm UTC.

> The anon key is *meant* to be public: it is already inside the app every member's phone has downloaded. It is a secret here only because GitHub has no other place to put it.

---

5. Switch on the weekly backup · 10 minutes

Two more secrets, and one of them is a real one.

1. In Supabase, go to **Project Settings → Database → Connection string**, and choose the **URI** tab. Copy the whole line. It begins `postgresql://`.
2. Make up a long passphrase, or have a password manager make one. **Put it in the password manager first.** If GitHub is the thing that has broken, this repository is not where you want the only copy.
3. Add both as repository secrets, the same way as step 4:

Name	What to paste
`SUPABASE_DB_URL`	The connection string from step 1
`BACKUP_PASSPHRASE`	The passphrase from step 2

1. **Actions → backup → Run workflow.** It takes a couple of minutes.
2. When it is green, open the run and download the file named **beacon-backup**.

Then restore it once, before you need to

A backup nobody has restored is a rumour. This has never been done.

```
gpg --decrypt --output beacon.sql beacon-<date>.sql.gpg
```

If that produces a file and does not ask twice, the backup is real and your passphrase works. Restoring it into a scratch Supabase project is the fuller test and is worth an hour one evening.

After that it runs every Sunday at 8pm UTC on its own.

---

6. Fill in the privacy notice · 30 minutes, then a lawyer

The app now has a notice at **/privacy**, reachable from Settings. It is a draft, and the parts that must be filled in are highlighted in amber on the page itself so they cannot be missed.

Open `app/privacy/page.tsx` and replace each `<Blank what="…" />`:

Blank	What it needs
`[church name and address]`	The legal name and address of whoever is responsible
`[name and email]`	Your Data Protection Officer, from step 7
`[retention period]`	Your answer to step 8
`[number]`	How many days you promise to answer a request in. 15 is a common choice

The hosting region is already filled in: **Seoul, South Korea**.

Then have somebody qualified read it. `docs/DATA-PROTECTION.md` is the map they will want, and it is written for exactly that purpose.

---

7. Name a Data Protection Officer · one decision

Under the Philippine Data Privacy Act, this app processes **sensitive** personal information, because a person's age and their religious affiliation both are and this app records both. That expects a named officer.

1. Pick a person. It does not have to be a lawyer, and in a small church it is usually whoever runs it.
2. Give them an email address that is not a personal one.
3. Put both into the notice in step 6.

If the church passes about a thousand members, registration with the National Privacy Commission at **privacy.gov.ph** is expected as well. You are at sixty-three, so this is a note for later rather than a task for today.

---

8. Decide how long things are kept · one meeting

Right now everything except the 30-day library record is kept because nothing deletes it, rather than because anybody chose. That is the honest position and it is not a good one.

Decide a period for each, and then tell me and I will make the app enforce it:

	A reasonable starting point
Messages in a conversation	While the pairing is active, plus a year
Files sent in a conversation	The same
Prayer requests	Two years
Meetings and follow-ups	Two years
Safeguarding reports	**Permanent, and this one should not change**
The record of removals	**Permanent, for the same reason**

Those are suggestions, not advice. What matters is that somebody chooses.

---

9. Pair the four Explorers who have nobody · this week

Four Explorers are approved and have no Guide. They can open the app and there is nothing in it for them.

1. Sign in as a Director.
2. **Admin → Pairings.**
3. The Explorers with no Guide are listed. You have twenty-nine Guides carrying twenty-four people between them, so there is room.

This is the only item on this page about somebody being ignored rather than about money or paperwork.

---

10. Point the domain, then redeploy · 20 minutes, then a wait

Your domain is bought and points at nothing.

1. In Vercel, open the project → **Settings** → **Domains** → **Add**, and enter your domain. Vercel will show you the records to create.
2. At whoever sold you the domain, add exactly those records.
3. Wait. Twenty minutes is usual; a few hours is normal.

Then, and only then

Set the app's own address so it can warn anybody running an old copy:

1. Vercel → **Settings** → **Environment Variables** → **Add**.
2. Name `CANONICAL_HOST`, value your domain (for example `yourchurch.org`), for Production.
3. **Redeploy.** This one is read when the app is built, so saving it changes nothing on its own.

> **Read this before you point the domain.** About thirty-five people have the app installed from its current address. A browser identifies an installed app by its address, so **every one of those copies stops updating the day you move**, and each of those people has to delete the icon and add it again.
>
> The app has a warning screen for exactly this, which is why step 2 above matters. But announce the move *before* you make it, not after.

---

11. The four failing tutorial tests · for me, not for you

Four end-to-end tests fail, on both browser engines, and were failing before any of this work started. Two of them say the guided tutorial's spotlight lands on nothing, which means **the tutorial is not safe to demonstrate** until it is looked at.

Nothing for you to do. It is on my list, and it is the reason the tutorial has not been part of any demo advice.

The order I would do them in

Today, twenty minutes: 1, 2, 3. You will know whether deploys work, what the hosting really costs, and you will be back inside budget.

This week: 4, 5, 9. The safety nets on, and four people paired.

This month: 6, 7, 8, 10. The paperwork, and the domain when you are ready to tell everybody about it.